



Testes E2E do Zero com Claude Code e Playwright

Uma jornada completa de desenvolvimento — do PRD à aplicação funcional, com testes E2E automatizados usando Claude Code e Playwright MCP.

Marcelo Guerra · 2026

Marcelo Guerra

Especialista em IA Aplicada, Software e Hardware

Engenharia de IA Aplicada

Aplicando inteligência artificial em soluções práticas e escaláveis.

Engenheiro Eletrônico

Base sólida em engenharia aplicada a sistemas complexos.

Projetos, Programas e Produtos

Gestão e entrega de produtos digitais de ponta a ponta.

Nosso Objetivo com Playwright

Nossa jornada com o Playwright foca em:

- **Exploração, Desenvolvimento e Testes:** Compreender a ferramenta do zero, suas capacidades e aplicações práticas.
- **Propósitos Educacionais:** O objetivo principal é o aprendizado e a experimentação, não a criação de aplicações prontas para produção. Iremos experimentar o uso do Playwright para testes e também como suporte ao desenvolvimento e documentação.
- **Planejamento Simplificado:** Abordaremos conceitos de arquitetura de software de forma descomplicada, essenciais para qualquer projeto.

5. Suíte 4 – Gestão de Tarefas (Todos)

> Pré-condição comum: usuário logado, lista "Tarefas da Sem

Cenário 4.1 – Criar tarefa via **slideover**

****ID:**** `TODO-001`

****Passos:****

1. Clicar no botão "Adicionar Tarefa" no header da lista.
2. Verificar abertura do **slideover** "Adicionar Tarefa".
3. Preencher "Título" com `Comprar pão na padaria`.
4. Selecionar "Complexidade" → ****Baixa****.
5. Selecionar "Estimativa para conclusão" → ****Horas****.
6. Selecionar tag "Compras" (do seed).
7. Deixar os campos do Fieldset "Tempo" vazios.
8. Clicar em "Criar".

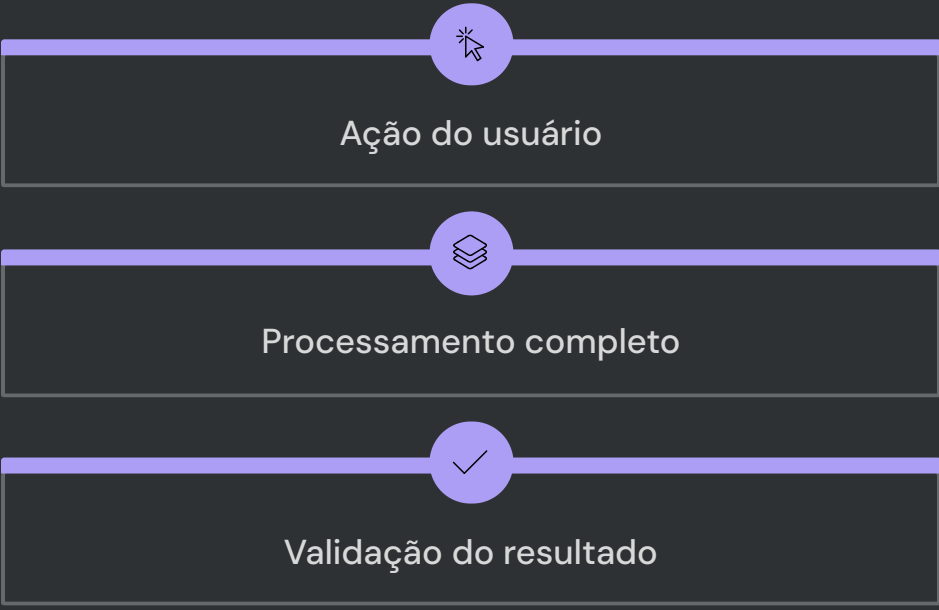
****Esperado:****

- Slideover fecha.
- Notification: "Tarefa adicionada".
- O novo item aparece ****no final**** da lista com:
 - badge verde "Baixa"
 - badge cinza "Horas"
 - badge azul "Compras"
 - sem datas de início/conclusão
 - sem strikethrough no título

END-TO-END

O que é um Teste End-to-End (E2E)?

Testes End-to-End (E2E) simulam cenários completos de uso, da interface até o banco de dados, garantindo que todos os componentes da aplicação funcionem de forma coesa.



Instalação do Ambiente

Stack Principal

1

Laravel 13 + SQLite

Framework PHP moderno com banco de dados leve, sem configuração extra.

2

Filament 5

Painel admin moderno com tabelas, formulários e widgets prontos.

3

Laravel Reverb

Broadcasting nativo para atualizações em tempo real via WebSockets.

4

Pest + Playwright

Framework de testes elegante e expressivo para PHP.

Comandos de Setup

```
laravel new playtask --git --database=sqlite  
--branch=main --livewire --pest --npm --boost
```

```
cd playtask
```

```
php artisan install:broadcasting  
--reverb --no-interaction --force
```

```
composer require filament/filament:"^5.0"  
php artisan filament:install --panels
```

```
rm -rf .git && git init  
git add . && git commit -m "Initial Commit"
```

```
clear  
export MAX_MCP_OUTPUT_TOKENS=120000  
export CLAUDE_CODE_MAX_OUTPUT_TOKENS=64000  
claude --dangerously-skip-permissions
```

PHPUnit vs PestPHP

PHPUnit

- O framework de testes tradicional e robusto para PHP.
- Sintaxe clássica baseada em asserções para validação (``assertEquals``).
- Configuração detalhada através de arquivos XML.
- Vasta comunidade e um ecossistema maduro.
- Ideal para projetos legados ou que preferem uma abordagem mais formal.

PestPHP

- Uma alternativa moderna, elegante e minimalista, construída sobre PHPUnit.
- Sintaxe fluente e expressiva baseada em expectativas (``expect()->toBe()``).
- Configuração concisa usando arquivos PHP.
- Foco na experiência do desenvolvedor e simplicidade.
- Integração nativa com Playwright para testes E2E.

PestPHP moderniza os testes PHP com uma sintaxe mais expressiva e integração nativa com Playwright — ideal para quem quer produtividade sem abrir mão de robustez.

Playwright

Framework de testes e automação baseado em Chromium, Firefox e Webkit

	Ideal para	Instalação
Playwright Test	Testes end-to-end	<code>npm init playwright@latest</code>
Playwright CLI	Agentes de código (Claude Code, Copilot)	<code>npm i -g @playwright/cli@latest</code>
Playwright MCP	Agentes de IA e automação com LLMs	<code>npx @playwright/mcp@latest</code>
Playwright Library	Scripts de automação de browser (utilizado pelo PestPHP)	<code>npm i playwright</code>
VS Code Extension	Criação e depuração de testes no VS Code	Instalar do Marketplace



Cross-browser testing that works. Everywhere.

- TypeScript
Python
Java
.NET
- Chrome
Edge
Safari
Firefox
- Windows
macOS
ubuntu



GitHub – microsoft/playwright: Playwright is a framework for Web Testing and Au...

Playwright is a framework for Web Testing and Automation. It allows testing Chromium, Firefox and WebKit with a single API. – microsoft/playwright



Playwright

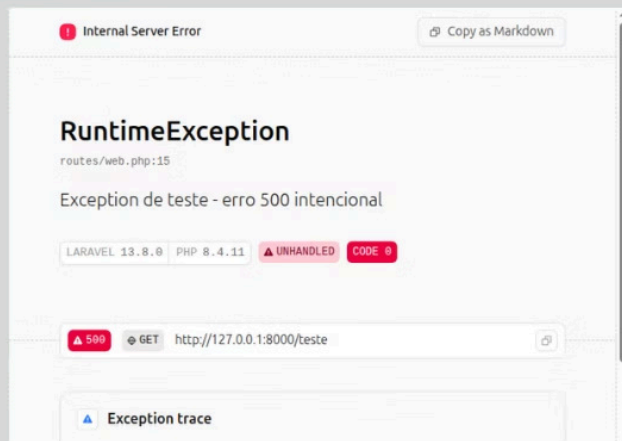
Playwright CLI

- Acesso completo a todas as funções do Playwright.
- Ideal para testes E2E funcionais e determinísticos.
- Fluxo de teste bem definido e previsível.
- Executa como comando com resposta determinística.
- Não consome tokens de IA — mais rápido e econômico.
- Daemon já em execução, sem overhead de inicialização.
- Imagens de página: salvas em arquivo

Playwright MCP

- Permite que a IA trabalhe em loops de pensamento e exploração.
- A IA inicia o servidor a cada nova instância, mas mantém a sessão.
- Decide e interpreta resultados a cada momento.
- Consome mais tokens e mais tempo de execução.
- Ideal para exploração, desenvolvimento e conversas com LLMs.
- Melhor usado em modo headed.
- Imagens de página: retorna inline como conteúdo multimodal

Para testes E2E, Playwright CLI em modo headless é mais eficiente e econômico. Para exploração e desenvolvimento em conversas com LLMs, Playwright MCP em modo headed é mais interessante.



RuntimeException - Erro 500

Resposta raw do browser_navigate

```
### Ran Playwright code
'''js
await page.goto('http://127.0.0.1:8000/teste');
'''

### Page
- Page URL: http://127.0.0.1:8000/teste
- Page Title: PlayTask
- Console: 1 errors, 0 warnings
### Snapshot
- [Snapshot](.playwright-mcp/page-2026-05-12T16-37-27-871Z.yml)
### Events
- New console entries: .playwright-mcp/console-2026-05-12T16-37-24-658Z.log#L1-L2
```

```
.playwright-mcp > console-2026-05-12T16-37-24-658Z.log
1 [ 540ms] [ERROR] Failed to load resource: the
server responded with a status of 500 (Internal Server
Error) @ http://127.0.0.1:8000/teste:0
2 [ 612ms] [LOG] Browser logger active (MCP
server detected). Posting to: http://127.0.0.1:8000/
boost/browser-logs @ http://127.0.0.1:8000/teste:21
3
```

.playwright-mcp/console.log

```
- generic [ref=e18]:
- generic [ref=e19]:
  - heading "RuntimeException" [level=1] [ref=e20]
  - generic [ref=e22]: routes/web.php:15
  - paragraph [ref=e23]: Exception de teste - erro 500
- generic [ref=e24]:
  - generic [ref=e25]:
    - generic [ref=e26]:
      - generic [ref=e27]: LARAVEL
      - generic [ref=e28]: 13.8.0
    - generic [ref=e29]:
      - generic [ref=e30]: PHP
      - generic [ref=e31]: 8.4.11
  - generic [ref=e32]:
    - img [ref=e33]
    - text: UNHANDLED
```

.playwright-mcp/page.yml

Desenvolvimento do Projeto



Os 5 Prompts do Projeto

Nossa abordagem de projeto é estruturada em cinco prompts sequenciais, guiando o desenvolvimento desde a concepção até a documentação final.

1 Prompt 1 — Criação do PRD

Definindo o planejamento do produto e os requisitos principais.

2 Prompt 2 — Planejamento e Implementação

Traduzindo o PRD em fases executáveis e implementando cada uma.

3 Prompt 3 — Documento de Testes E2E

Criando um documento completo de testes E2E mapeando os principais fluxos e papéis de usuário.

4 Prompt 4 — Execução com Playwright MCP

Executando os testes definidos com Playwright MCP para validação completa.

5 Prompt 5 — Refatoração e Documentação

Refatorando a Landing Page e gerando documentação com screenshots automatizados via Playwright.

Prompt 1 — Criação do PRD

Geração do documento `docs/prd.md` para a aplicação **PlayTask**, definindo toda a arquitetura, painéis e regras de negócio.

Guest Panel

Landing page e todo lists públicas, sem autenticação.

Admin Panel

Painel do usuário para criar, gerir e manter todo lists. Sidebar collapsible.

Superadmin Panel

Painel SaaS para visualizar e-mails do Beta e criar usuários com senha aleatória.



Gestão de Todo Lists

Sidebar com listas ordenadas por `created_at`. Criação e edição via slideover. Delete com confirmação pelo nome do item.



Tempo Real com Reverb

Tudo lists, itens e status atualizados em tempo real no Admin Panel e nas páginas públicas via Laravel Reverb + Echo.



Controle de Acesso

Listas públicas ou privadas, com suporte a senha (hash bcrypt). Password wall no primeiro acesso, sem reautenticação na sessão.



Dashboard com Widgets

Gráfico de pizza para listas concluídas, atividades por tags e atividades iniciadas/concluídas por dia (últimos 30 dias).



Commit: `"prompt 1 - criação do prd"`

Prompt 2 — Planejamento e Implementação

A partir do `docs/prd.md`, o agente cria e prioriza todas as atividades necessárias para implementar a aplicação, divididas por fases.

1

Leitura do PRD

Análise e priorização das atividades por fases

2

Implementação

Uso de comandos Artisan sempre que possível

3

Seeders & Testes

Seeders para todos os dados + testes Pest para fluxos principais

4

Documentação

`.env.example` com chaves Reverb + `readme.md` com `composer dev`

✔ Commit: "prompt 2 - do prd à implementação"

Prompt 3 — Documento de Testes E2E

Análise da aplicação e geração do documento `docs/e2e-testing.md` com todos os principais fluxos de uso dos usuários.

1 Login e autenticação

Fluxo completo de acesso ao Admin Panel

2 Criação de todo lists

Criação, edição e configuração de slugs

3 Itens e atividades

Criação, edição e conclusão de atividades

4 Páginas públicas

Visualização com e sem senha

Objetivo do Documento

- Garantir que todos os fluxos críticos estejam cobertos antes da execução automatizada com Playwright MCP.
- O teste do Reverb valida que o servidor está ativo e que as atualizações em tempo real funcionam corretamente no browser.

📄 Commit: `"prompt 3 - plano de testes e2e"`

Prompt 4 — Execução dos Testes com Playwright MCP

Utilização do **Playwright MCP** para executar todos os fluxos detalhados em `docs/e2e-testing.md`, com captura de screenshots e resolução de problemas em tempo real.



Screenshots de Todas as Telas

Capturas de todas as telas populadas, salvas em `resources/images` com nomes descritivos.



Resolução de Problemas

Problemas encontrados durante a execução são resolvidos com foco em features, UI e UX.



Validação Completa

Todos os fluxos do documento E2E são executados e validados pelo agente de forma autônoma.



Commit: `"prompt 4 - execução de testes e2e"`

Prompt 5 — Refatoração da LP e Documentação

Refatoração da Landing Page

Usando as imagens capturadas nos testes E2E, a landing page é refatorada seguindo padrões de mercado para SaaS.

Novo item de menu **Docs** com página de documentação navegável

Uso das imagens E2E, fluxo documentado e conhecimento geral do produto

Novos testes E2E verificando LP, página de docs, imagens e textos

Validação de paddings, margens, botões, menus e actions

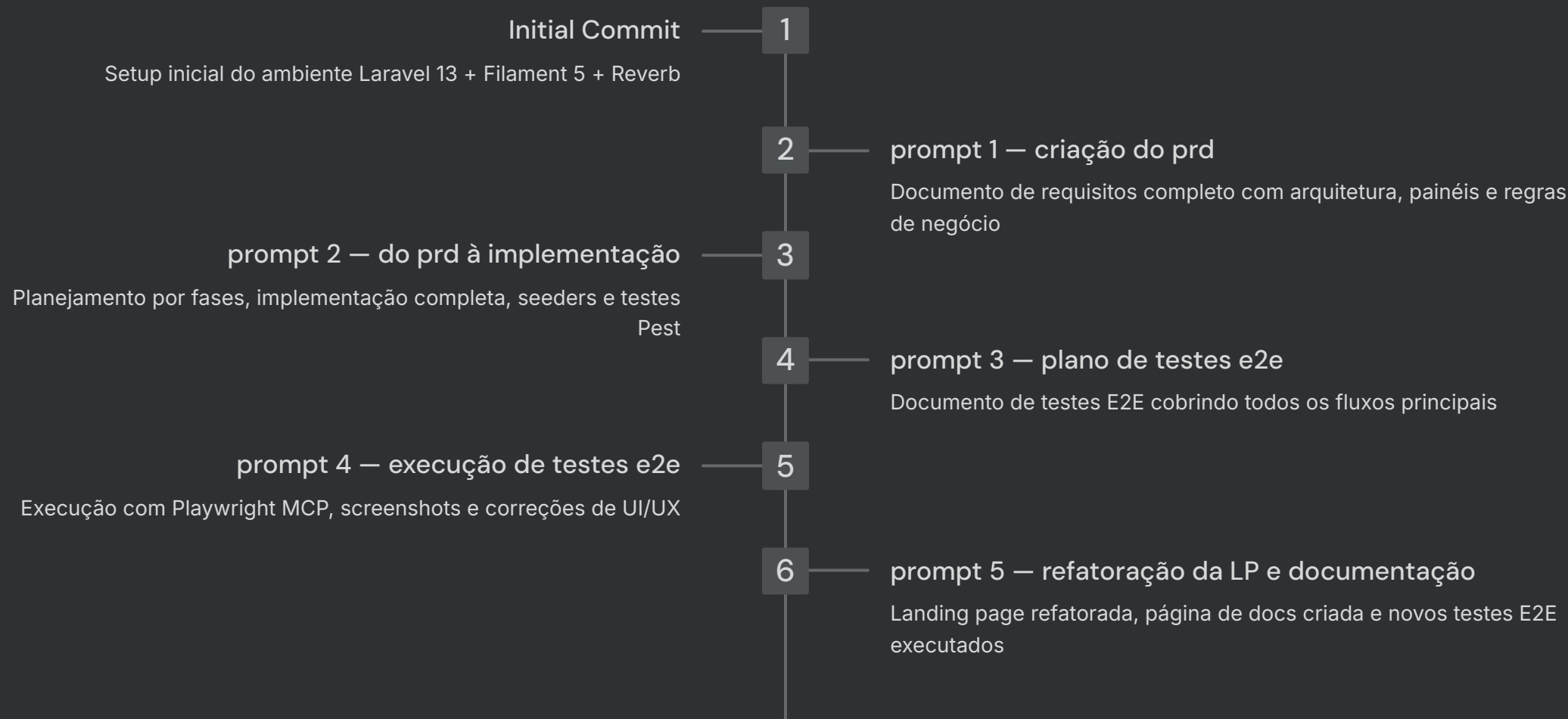
Critérios de Qualidade

- Imagens e textos carregando corretamente
- Paddings e margens seguindo boas práticas
- Botões 100% funcionais
- Menus e actions operacionais
- Documentação simples e navegável

❏ Commit: "prompt 5 - refatoração da landing page e criação da página de documentação"

Histórico de Commits

Toda a evolução do projeto registrada em 5 prompts, do zero à aplicação completa com testes E2E.



Obrigado!



GitHub

github.com/mwguerra



Website

mwguerra.com



LinkedIn

linkedin.com/in/marcelowguerra

Vamos conversar sobre como a Engenharia de IA Aplicada pode transformar o seu processo de desenvolvimento.